

Trabajo Práctico N.º 2

Componentes, Props y Estado con useState

Tema: Componentes, props y estado

Modalidad: Individual

Herramientas: React + Vite con TypeScript

1. Objetivo

Practicar el uso del hook useState para manejar el estado de un componente, tipar correctamente los datos con TypeScript, y aprender a comunicar información entre componentes mediante props.

2. Contexto

Vas a construir una app con tres componentes. Cada uno tiene una responsabilidad específica y bien diferenciada:

<code>App.tsx</code>	Componente padre. Tiene los dos estados y toda la lógica. Les pasa los datos a sus hijos.
<code>StudentList.tsx</code>	Componente intermedio. Recibe la lista filtrada y renderiza una <code>StudentCard</code> por cada estudiante.
<code>StudentCard.tsx</code>	Componente de presentación. Recibe un único estudiante y muestra su información.

3. ¿Qué debe hacer la app?

La app debe permitirle al usuario:

1. Ver la lista vacía al iniciar.
2. Presionar un botón **"Cargar estudiantes"** para que aparezcan los datos. El array de estudiantes ya debe estar definido en el código, no se usa ninguna API ni fetch.
3. Escribir en un **input de texto** para filtrar por nombre en tiempo real. La lista se actualiza con cada tecla que el usuario presiona.

4. ¿Qué estados debe tener App.tsx?

El componente App debe manejar exactamente dos estados con useState:

Estado 1 — El texto que el usuario está escribiendo en el input.

Estado 2 — El array de estudiantes que se muestra actualmente (empieza vacío).

5. Tipado con TypeScript

Al trabajar con TypeScript es necesario definir el tipo de los datos que se manejan. Para eso vas a crear una interfaz que describa la forma de un estudiante.

La interfaz debe llamarse

Student

y debe incluir al menos estas propiedades:

- : — identificador único del estudiante
- : — nombre completo
- : — número de legajo

Podés agregar otras propiedades opcionales si querés (carrera, año, etc.).

Importante: la interfaz debe usarse para tipar los estados en App.tsx, las props en StudentList.tsx y las props en StudentCard.tsx. No está permitido usar .

6. Props de cada componente

Cada componente hijo debe definir sus props usando TypeScript. Tené en cuenta lo siguiente:

- recibe un array de estudiantes ya filtrado (tipo).
- recibe un único estudiante (tipo).
- Ninguno de los dos componentes hijos tiene estado propio ni lógica de filtrado.

7. Requisitos mínimos

- Tres archivos de componentes: App.tsx, StudentList.tsx y StudentCard.tsx.
- Una interfaz Student definida y reutilizada en los tres componentes.
- No se permite usar any en ningún lugar del código.
- Los datos deben estar definidos como un array en el código, sin fetch.
- Al iniciar la app la lista debe estar vacía.
- El botón "Cargar estudiantes" debe poblar la lista.
- El filtro por nombre debe funcionar en tiempo real.
- StudentList renderiza una StudentCard por cada estudiante del array que recibe.
- La lista mínima debe tener 6 estudiantes.

8. Pistas para arrancar

Tip 1 — El array completo de estudiantes puede ser una constante fuera del componente, no necesariamente un estado. Lo que sí es estado es cuáles se muestran en este momento.

Tip 2 — El filtrado lo hace App antes de pasarle los datos a StudentList. Pensá qué método de array te permite quedarte solo con los elementos que cumplen una condición.

Tip 3 — Para definir las props de un componente en TypeScript podés usar una interfaz con el nombre Props o usar directamente el tipo en los parámetros de la función.

Tip 4 — Para recorrer un array y renderizar un componente por cada elemento, React usa el método map(). Acordate de que cada elemento renderizado necesita una prop key única.

9. Criterios de evaluación

Criterio	Puntaje
Interfaz Student definida y usada en los tres componentes sin any	25 pts
Los dos estados en App están correctamente tipados y funcionan	20 pts
El botón carga los estudiantes y el filtro funciona en tiempo real	20 pts

StudentList usa map() para renderizar una StudentCard por elemento	20 pts
El código está limpio y es fácil de leer	15 pts